

Western Norway
University of
Applied Sciences

Rounding data and Dynamic Programming

Chapter 3

Sven-Olai Høyland

Dynamic Programming (DP)

- Seen how to use DP to find optimal solution
  - Build optimal solution from optimal solution of subproblems (often stored in table/multidimensional arrays)
  - We used DP in the Longest Common Subsequence problem

- Approximations algorithms can also use DP
  - Round input data
  - Scheduling problems
    Make distinctions between “large” and “small” elements

Knapsack Problem

- Set of items $I = \{1,2, \dots, n\}$
- The knapsack has a capacity $B > 0$
- Each item i has a value $v_i \geq 0$ and a size $0 \leq s_i \leq B$
- Goal
  Find a subset of the items $S \subseteq I$ that maximizes $\sum_{i \in S} v_i$ where $\sum_{i \in S} s_i \leq B$

DP to find optimal solution

- Maintain array $A(j)$ for $j = 1,2, \ldots n$
- Each entry $A(j)$ is a list of pairs (t, w).
- A pair (t, w) in the list of entry $A(j)$ indicates that there exists a set $S \subseteq \{1,2, \ldots, j\}$, such that
  - $\sum_{i \in S} s_i = t \leq B$ and
  - $\sum_{i \in S} v_i = w$
- Not all pairs, just the most efficient (next slide)

Dominating pair

- A pair (t, w) dominates another pair (t', w') if $t \leq t'$ and $w \geq w'$
- Domination is a transitive property
- Assure that no pair dominates another one. Then we can assume that each list $A(j)$ is of the form (t_1, w_1),…, (t_k, w_k), where $t_1 < t_2 < \cdots < t_k$ and $w_1 < w_2 < \cdots < w_k$
- At most $B + 1$ pair in each list
- Ensure that $A(j)$ contains a pair that dominates each feasible set $S \subseteq \{1, \ldots, j\}$

DP algorithm

$\mathrm{A}(1) \leftarrow \{(0,0), (s_1, w_1)\}$

for j $\leftarrow 2$ to n do

    $\mathrm{A}(j) \leftarrow A(j-1)$

    for each $(t, w) \in A(j-1)$ do

        if $t + s_j \leq B$ then

            $\mathrm{A}(j) \leftarrow A(j) \cup \{(t + s_j, w + v_j)\}$

    remove dominated pairs from $A(j)$

return $\max_{(t,w) \in A(n)} w$

Example

Set of items (size, value)
$I = \{(2,3), (4,3), (3,4), (3,3), (4,7)\}$

$B = 7$

Find

$A(1) = \{(0,0), (2,3)\}$
$A(2) = ?$

$A(2) = ?$ (after removing dominating pairs)
$A(3) = ?$

$A(3) = ?$ (after removing dominating pairs)
$A(4) = ?$

$A(4) = ?$ (after removing dominating pairs)
$A(5) = ?$

$A(5) = ?$ (after removing dominating pairs)

Answer?

Solution (filled in during class)

$A(1) = \{(0,0), (2,3)\}$
$A(2) = \{(0,0), (2,3), (4,3), (6,6)\}$
$A(2) = \{(0,0), (2,3), (6,6)\}$
$A(3) = \{(0,0), (2,3), (6,6), (3,4), (5,7)\}$
$A(3) = \{(0,0), (2,3), (3,4), (5,7)\}$
$A(4) = \{(0,0), (2,3), (3,4), (5,7), (3,3), (5,6), (6,7)\}$
$A(4) = \{(0,0), (2,3), (3,4), (5,7)\}$
$A(5) = \{(0,0), (2,3), (3,4), (5,7), (4,7), (6,10), (7,11)\}$
$A(5) = \{(0,0), (2,3), (3,4), (4,7), (6,10), (7,11)\}$

Answer? 11



Pseudopolynomial

- Definition 3.2
  - An algorithm for a problem Π is said to be pseudopolynomial if its running time is polynomial in the size of the input when the numeric part of the input is encoded in unary
  - Example 5 coded unary is 11111

Poly-time approximation schemes

- Definition 3.3 (also given in Chapter 1)
  - A polynomial-time approximation scheme (PTAS) is a family of algorithms $\{A_\varepsilon\}$ where there is an algorithm for each $\varepsilon > 0$, such that A_ε is a (1+ε)-approximation algorithm (for minimization problems) or a (1-ε)-approximation (for a maximization problems)

  - Smaller $\varepsilon \rightarrow$ longer time

Definition 3.4

- A fully polynomial-time approximation scheme (FPAS, FPTAS) is an approximation scheme such that the running time of A_ε is bounded by a polynomial in $\frac{1}{\varepsilon}$.

FPTAS for Knapsack

- Measure value in integer multiples of μ (to reduce the number of possible values)
- Convert each value $v_i' = \lfloor v_i/\mu \rfloor$.
- Run DP algorithm for knapsack with converted values
- Each feasible solution has changed with a most $n\mu$
- $M = \max_{i \in I} v_i$ is a lower bound on OPT
- Set $n\mu = \varepsilon M$ which gives $\mu = \frac{\varepsilon M}{n}$

Running time

- Modified values
  $V' = \sum_{i=1}^{n} v_i' = \sum_{i=1}^{n} \left\lfloor \frac{v_i}{\varepsilon M/n} \right\rfloor = \frac{n}{\varepsilon M} \sum_{i=1}^{n} v_i \leq \frac{n^2}{\varepsilon M}$ which is $O\left(\frac{n^2}{\varepsilon}\right)$

- Running time: $O(n \min(B, V'))$ which is $O(n^3/\varepsilon)$
  - Bounded by a polynomial in $\frac{1}{\varepsilon}$
- Have to prove that algorithm returns solution whose value is at least $(1 - \varepsilon)OPT$

Fully Approximation scheme

For knapsack

$M \leftarrow \max_{i \in I} v_i$

$\mu \leftarrow \varepsilon M / n$

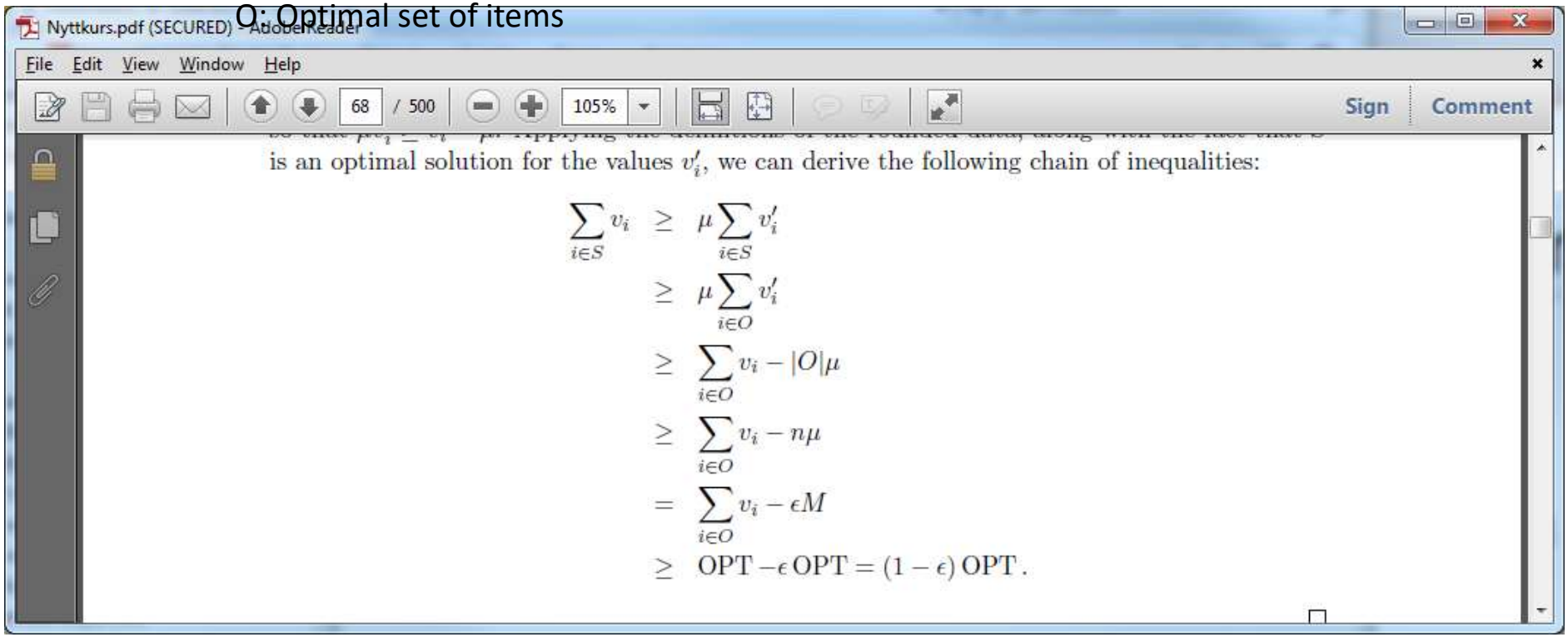
$v_i' = \lfloor v_i / \mu \rfloor$ for all $i \in I$

Run algorithm for DP knapsack algorithm

Quality of solution

S: Set of items returned by the algorithm

O: Optimal set of items



is an optimal solution for the values v'_i, we can derive the following chain of inequalities:

$$\begin{aligned}
\sum_{i\in S} v_i &\geq \mu \sum_{i\in S} v'_i \\
&\geq \mu \sum_{i\in O} v'_i \\
&\geq \sum_{i\in O} v_i - |O|\mu \\
&\geq \sum_{i\in O} v_i - n\mu \\
&= \sum_{i\in O} v_i - \epsilon M \\
&\geq \mathrm{OPT} - \epsilon\,\mathrm{OPT} = (1-\epsilon)\,\mathrm{OPT}\,.
\end{aligned}$$

Scheduling jobs

- n jobs to be processed
- m identical machines (running in parallel) to which each job may be assigned
- Each job $1, \ldots, n$ must be processed on one of these machines for p_j time units without interruption
- Each job is available at time 0
- Each machine can process at most one job at the time
- The aim is to complete all jobs as soon as possible

Algorithm (1)

- Let k be a fixed positive integer
- Derive a family of algorithms
- Main idea
  - Partition jobs in two sets
    - Long jobs
    - Short jobs. A job ℓ is short if

$$p_\ell \leq \frac{1}{km} \sum_{j=1}^{n} p_j$$

  - At most km long jobs

Algorithm (2)

- Enumerate all possible schedules for the long jobs, and chose one with minimum makespan.
- Extend this schedule by using list scheduling for the short jobs by assigning next job to the machine least loaded.

Quality of solution

- Focus on the last job to finish, ℓ.
- If it is a long job, we have optimal solution
- Otherwise, we have

Job ℓ is short

  - $C_{max} \leq p_\ell + \sum_{j \neq \ell} p_j / m$
- Now
  - $C_{max} \leq \sum_{j=1}^{n} \frac{p_j}{mk} + \sum_{j \neq \ell} \frac{p_j}{m} \leq \left(1 + \frac{1}{k}\right) \sum_{j=1}^{n} \frac{p_j}{m} \leq \left(1 + \frac{1}{k}\right) C^*_{max}$

Theorem 3.6

- The family of algorithms $\{A_k\}$ is a polynomial-time approximation scheme for the problem of minimizing the makespan on any constant number of identical parallel machines.

Theorem 3.7

- There is a polynomial-time approximation scheme for the problem of minimizing the makespan on any number of identical parallel machines.

Extend the results

- We considered the number of machines to be a constant
- Possible to extend the result such that the number of machines is part of the input

The Bin-Packing Problem

- Given n pieces (or items) with specified sizes $a_1, a_2, \ldots, a_n$ such that $1 \geq a_1 \geq a_2, \geq \cdots \geq a_n > 0$
- Wish to pack the items into minimum number of bins such that the sum of sizes in each bin is at most 1.

Partition Problem

- Given n positive integers $b_1, b_2, \ldots, b_n$ whose sum $B = \sum_{i=1}^{n} b_i$ is even.
- Wish to know if we can partition the set of indices $\{1,2, \ldots, n\}$ into sets S and T such that $\sum_{i \in S} b_i = \sum_{i \in T} b_i$
- This is known NP-complete problem

Theorem 3.8

- Unless P=NP, there cannot exist a ρ-approximation algorithm for the bin-packing problem for any $\rho < 3/2$.
- Proof
  If we have a Partition problem, we can forumulate the corresponding bin-packing problem

  Set $a_i = \frac{2b_i}{B}$ and check whether we can pack all the pieces into two bins or not.

First-Fit-Decreasing

- Pieces packed in order of non-increasing size
- Next piece is always packed into the first bin in which it fits

- If FFD(I) denotes the number of bins used by this algorithm on input I, and OPT(I) denotes the number of bins used in the optimal packing, then

Aditive constant

  - $FFD(I) \leq \frac{11}{9} OPT(I) + 4$ for any input I

Example

- .60, .55, .50, .43, .35, .33, .30, .30, .30, .25, .18

First-Fit

- As First-Fit-Decreasing without ordering the pieces first.
- Pair up bins 1 & 2, 3 & 4, and so on
- Each pair must have total piece size at least 1. First item in bin 2k did not fit in bin 2k-1
- FF use ℓ bins
- Let $SIZE(I) = \sum_{i=1}^{n} a_i \geq \lfloor \ell/2 \rfloor$
- Not possible to pack a bin more than full: $\mathrm{OPT(I)} \geq SIZE(I)$
- $FF(I) = \ell \leq 2SIZE(I) + 1 \leq 2OPT(I) + 1$

Family of algorithms

- Family of polynomial-time approximation algorithms parameterized by $\varepsilon > 0$
- Performance guarantee.
  The algorithms will use at most $(1 + \varepsilon)OPT(I) + 1$ bins for each input I

Will not have PTAS because of additive constant

Definition

- Because of the additive constant, we define
- Definition 3.9. An asymptotic polynomial-time approximation scheme (APTAS) is a family of algorithms $\{A_\varepsilon\}$ along with a constant c where there is an algorithm A_ε for each $\varepsilon > 0$ such that A_ε returns a solution of value at most $(1+\varepsilon)OPT + c$ for minimization problems.

Dynamic Programming

- Constant number of distinct piece sizes
- Constant number of pieces can fit into one bin



- Can solve bin-packing by dynamic programming in polynomial time

Can ignore small pieces

- Lemma 3.10. Any packing of all pieces of size greater than γ into ℓ bins can be extended to a packing for the entire input with at most $\max(\ell, \frac{1}{1-\gamma} SIZE(I) + 1)$ bins.

Idea

- Try to design algorithm with performance guarantee better than FF, $(1+\varepsilon)$ where $\varepsilon < 1$,
- Lemma 3.10 with $\gamma = \varepsilon/2$
- Then since $\frac{1}{1-\frac{\varepsilon}{2}} \leq 1+\varepsilon$, we see that the composite algorithm produces a packing with at most $\max(\ell, (1+\varepsilon)OPT(I)+1)$
- If each piece has size greater than $\varepsilon/2$, then each bin must contain fewer than $2/\varepsilon$ pieces in total

Linear grouping

- Group the pieces of the given input I as follows: The first group consists of the k largest pieces, the next group consists of the next the k largest pieces and so on. The last group consists of h pieces, where $h \leq k$.
- The rounded instance I' is constructed by discarding the first group and for each other group rounding its size of the pieces up to the size of the largest piece.

Linear grouping

- Example
  - k = 4
  - Items, I:
    - 0.87, 0.80, 0.75, 0.75, 0.70, 0.68, 0.63, 0.63, 0.60, 0.55, 0.55, 0.53, 0.50, 0.50, 0.48, 0.45, …
  - Items, I’
    - 0.70, 0.70 0.70, 0.70, 0.60, 0.60, 0.60, 0.60, 0.50, 0.50, 0.50, 0.50, …

Lemma 3.11

- Let I’ be the input obtained from an input I by linear grouping with group size k. Then $OPT(I') \leq OPT(I) \leq OPT(I') + k$ and furthermore, any packing of I’ can be used to generate a packing of I with at most k additional bins.

Proof

- $OPT(I') \leq OPT(I)$
  - Consider the optimal packing of I. Pack the k largest items in I' where the k largest in items in I are packed, then the second group where the next k largest are packed in I.
- $OPT(I) \leq OPT(I') + k$
  - Show how we can obtain a packing of I from a packing of I'
  - Pack each of the k largest items in I in its own bin
  - Next k items in I where the k largest in I' and so on
  - Good packing for I' $\rightarrow$ "almost-as-good" packing for I

APTAS for Bin-Packing

- Consider I’
- Number of distinct piece sizes n/k where n is the number of pieces in I
- No small pieces (no piece of size smaller than $\varepsilon/2$)
  - $SIZE(I) \geq \varepsilon n/2$
- Set $k = \lfloor \varepsilon SIZE(I) \rfloor$
- Use dynamic programming algorithm from Section 3.2 (scheduling jobs)

Theorem 3.12

- For any $\varepsilon > 0$, there is a polynomial time algorithm for the bin-packing problem that computes a solution with at most $(1 + \varepsilon)OPT(I) + 1$ bins, that is, there is an APTAS for the bin-packing problem.
- Possible to improve both running time and the performance guarantee.